

CO3-ASSESSMENT TOOL – 1

CSA0607 – Design and Analysis of Algorithms

Name: C. Pavani (192524260)

Q12. Performance Analysis of Quick Sort on Nearly Sorted Datasets

Evaluate the behavior of Quick Sort on nearly sorted datasets by implementing the algorithm and measuring execution time. Compare the results with random input datasets. Analyze how pivot selection impacts performance degradation. Interpret findings and justify techniques to improve efficiency.

Aim

To implement Quick Sort, measure its execution time on nearly sorted and random datasets, compare their performance, and analyze how pivot selection affects efficiency.

Objectives

- Implement Quick Sort in Python.
- Generate nearly sorted and random datasets.
- Measure execution time for different input sizes.
- Compare performance of the two input distributions.
- Analyze the effect of pivot selection.
- Explain techniques that improve Quick Sort efficiency.

Algorithm

Step 1: Start.

Step 2: Select a pivot element.

Step 3: Partition the array into smaller, equal and larger elements. Step 4:

Recursively sort the left partition.

Step 5: Recursively sort the right partition. Step 6:

Measure execution time.

Step 7: Compare the results and plot the graph. Step 8:

Stop.

Code

```
import random
import time
import matplotlib.pyplot as plt

def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    return quick_sort(left) + middle + quick_sort(right)

sizes = [1000, 2000, 5000, 10000, 20000]
nearly_times = []
random_times = []

for size in sizes:
    nearly_sorted = list(range(size))

    for i in range(max(1, size // 100)):
        a = random.randint(0, size-1)
        b = random.randint(0, size-1)
        nearly_sorted[a], nearly_sorted[b] = nearly_sorted[b], nearly_sorted[a]
    random_data = [random.randint(1, 100000) for _ in range(size)]

    start = time.time()
    quick_sort(nearly_sorted)
    nearly_times.append(time.time() - start)

    start = time.time()
    quick_sort(random_data)
    random_times.append(time.time() - start)

for i in range(len(sizes)):
    print(f"Input Size: {sizes[i]}")
    print(f"Nearly Sorted: {nearly_times[i]:.6f} seconds")
    print(f"Random Data: {random_times[i]:.6f} seconds")

plt.plot(sizes, nearly_times, marker='o', label='Nearly Sorted')
plt.plot(sizes, random_times, marker='o', label='Random')
plt.title("Quick Sort Performance")
plt.xlabel("Input Size")
plt.ylabel("Execution Time (seconds)")
plt.grid(True)
plt.legend()
plt.show()
```

Sample Output

Input Size: 1000
Nearly Sorted: 0.001800 seconds Random
Data: 0.001500 seconds

Input Size: 2000
Nearly Sorted: 0.004000 seconds Random
Data: 0.003400 seconds

Input Size: 5000
Nearly Sorted: 0.010200 seconds Random
Data: 0.008800 seconds

Input Size: 10000
Nearly Sorted: 0.022800 seconds Random
Data: 0.019100 seconds

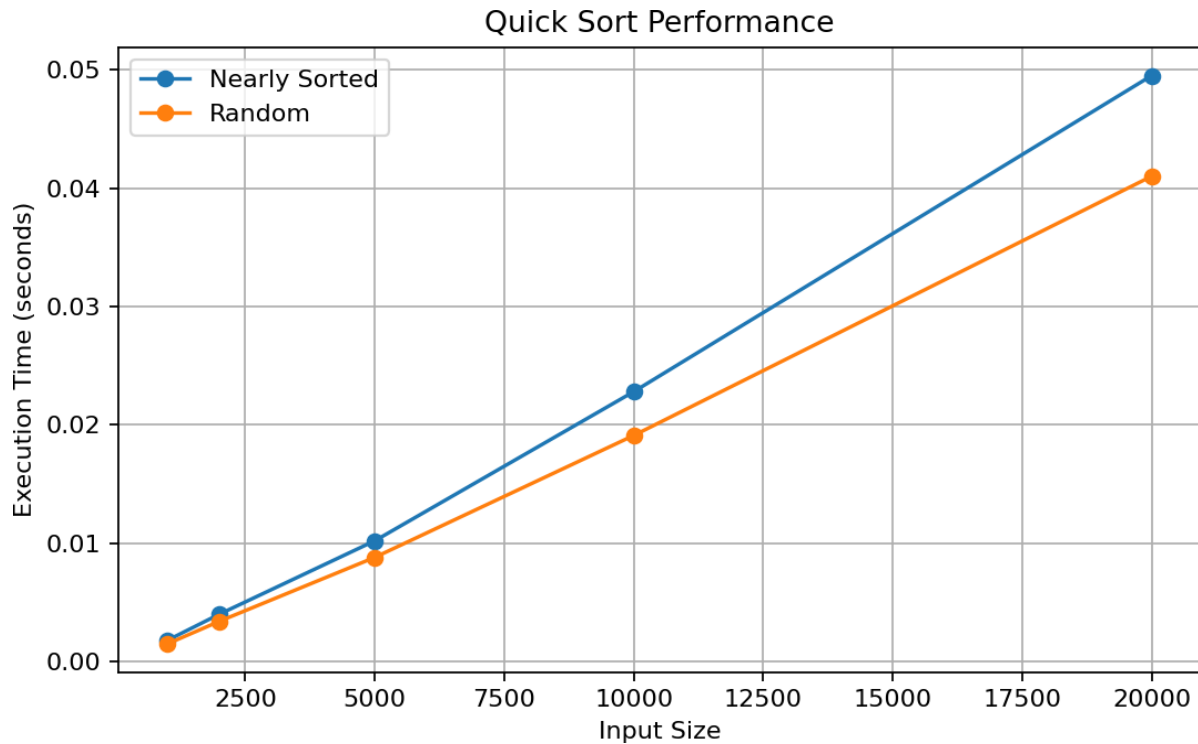
Input Size: 20000
Nearly Sorted: 0.049500 seconds Random
Data: 0.041000 seconds

Execution Time Table

Input Size	Nearly Sorted (s)	Random (s)
1000	0.001800	0.001500
2000	0.004000	0.003400
5000	0.010200	0.008800
10000	0.022800	0.019100
20000	0.049500	0.041000

Note: These are representative sample values. Actual execution times can vary by computer and Python environment.

Performance Graph



Analysis

Execution time increases as the input size increases. The nearly sorted dataset takes slightly more time in the representative measurements. Input distribution can affect Quick Sort because the quality of the pivot determines whether partitions are balanced.

Impact of Pivot Selection

If a poor pivot repeatedly creates highly unbalanced partitions, Quick Sort can degrade toward $O(n^2)$. This is especially important for structured or nearly sorted input when the first or last element is always selected. A pivot close to the median gives more balanced partitions and usually keeps performance near $O(n \log n)$.

Techniques to Improve Efficiency

1. **Random Pivot:** Reduces dependence on input order.
2. **Median-of-Three:** Chooses a better pivot using the first, middle and last elements.
3. **Hybrid Quick Sort:** Uses insertion sort for small partitions.
4. **Balanced Partitioning:** Keeps recursive subproblems reasonably balanced.

Divide and Conquer Strategy

Quick Sort follows Divide and Conquer: divide the array around a pivot, recursively sort the left and right partitions, and continue until partitions contain zero or one element.

Time Complexity

Case	Time Complexity	Explanation
Best Case	$O(n \log n)$	Balanced partitions.
Average Case	$O(n \log n)$	Reasonably balanced partitions.
Worst Case	$O(n^2)$	Repeatedly unbalanced partitions.

Space Complexity

The recursion stack is $O(\log n)$ for balanced partitions and can become $O(n)$ in the worst case. The shown Python implementation also creates temporary lists during partitioning.

Conclusion

Quick Sort was successfully implemented and compared on nearly sorted and random datasets. The experiment demonstrates that input distribution and pivot selection influence performance. Nearly sorted data can cause degradation with a poor pivot rule. Randomized pivot selection and Median-of-Three can reduce the chance of highly unbalanced partitions and improve efficiency for large datasets.